

Why there is a need of standard libraries?

A standard library is a collection of pre written, tested and optimized code that comes bundled with a programming language

Instead of re inventing the wheel, developers use these libraries to build applications faster and more reliable

1. Saves Development time

- Without standard libraries → you would have to write basic features yourself (like sorting, file I/O, data structures)

- With libraries → you just call a method

```
Arrays.Sort(arr); // instead of writing your own  
Sorting algorithm
```

2. Reliable and Well tested

- Standard libraries are Built and tested By language Maintainers
- Less chance of Bugs compared to writing own version

3. Consistency & Portability

- Code using standard libraries behaves the same across platforms
- Example: Java's FileReader works on Windows, Linux, Mac without rewriting the code

4. Performance Optimizations

- Standard libraries are usually faster & memory-efficient than custom implementations
- Example : String Builder in Java is optimized for concatenation compared to `+`.

5. Readability & Maintainability

- Developers immediately understand what `collection.sort()` or `Math.sqrt()` does
- if you write your own `mySpecialSquareRoot()`, future developers may waste time understanding / debugging

6. Industry Standards

- Using standard libraries ensures your code follows best practices

We need standard libraries because they are

- Time - Saving
- Reliable
- Portable
- Optimized
- Easy to understand
- Industry standard.

◆ Standard Libraries vs No Standard Libraries		
Aspect	With Standard Libraries	Without Standard Libraries
Development Speed	🚀 Very fast → just call methods (e.g., <code>Arrays.sort(arr)</code>).	🐢 Slow → must write everything from scratch (sorting, I/O, etc.).
Reliability	✅ Well-tested by experts (Oracle, Python Foundation, etc.).	❌ Risk of bugs due to custom, untested code.
Performance	⚡ Highly optimized for speed & memory.	🔴 May be slower unless you spend time optimizing.
Portability	🌐 Works across platforms (Windows, Mac, Linux).	⚠️ Needs platform-specific handling (e.g., file paths, networking).
Readability	👥 Other developers instantly understand (<code>Collections.sort()</code> is obvious).	😞 Custom methods (<code>mySortAlgo()</code>) may confuse future developers.
Maintainability	🔧 Easier to maintain; follows industry conventions.	👁️ Harder to maintain; logic may be duplicated everywhere.
Security	🔒 Many standard libraries handle edge cases & security issues (e.g., SQL escaping).	🔴 Higher risk of security flaws in custom implementations.
Learning Curve	📖 Teaches best practices (object-oriented design, error handling, etc.).	📖 May lead to bad practices & reinventing wheels.

Example: Sorting an Array

With Standard Library:

```
java
import java.util.Arrays;

int[] arr = {5, 2, 8, 1, 3};
Arrays.sort(arr); // One line!
```

Without Standard Library (write your own bubble sort):

```
java
int[] arr = {5, 2, 8, 1, 3};
for (int i = 0; i < arr.length - 1; i++) {
    for (int j = 0; j < arr.length - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

☞ Clearly, the standard library saves time, reduces errors, and makes the code cleaner.

shows the use of
standard libraries

How to Learn a Standard Library

✓ 1. Start with the Documentation

- Every language has **official docs** (most trusted resource).
- Java → [Java SE API docs](#)
- Python → [Python Standard Library docs](#)
- C++ → [cppreference](#)
- Pick **one package/module** at a time (e.g., in Java: `java.util`, `java.io`, `java.time`).

✓ 2. Learn by Categories, Not Everything

The standard library is huge. Focus on what you need:

- **Collections & Data Structures** → `ArrayList`, `HashMap`, `HashSet`.
- **Math & Utilities** → `Math`, `Random`, `BigInteger`.
- **I/O and Files** → `File`, `Scanner`, `BufferedReader`.
- **Date/Time** → `LocalDate`, `LocalTime`, `DateTimeFormatter`.
- **Concurrency (later)** → `Thread`, `ExecutorService`.

☞ Don't try to memorize. Instead, **understand categories and usage patterns**.

✓ 3. Read Examples + Run Code

Docs can feel abstract. Always combine reading with coding:

Example → Java `HashSet`

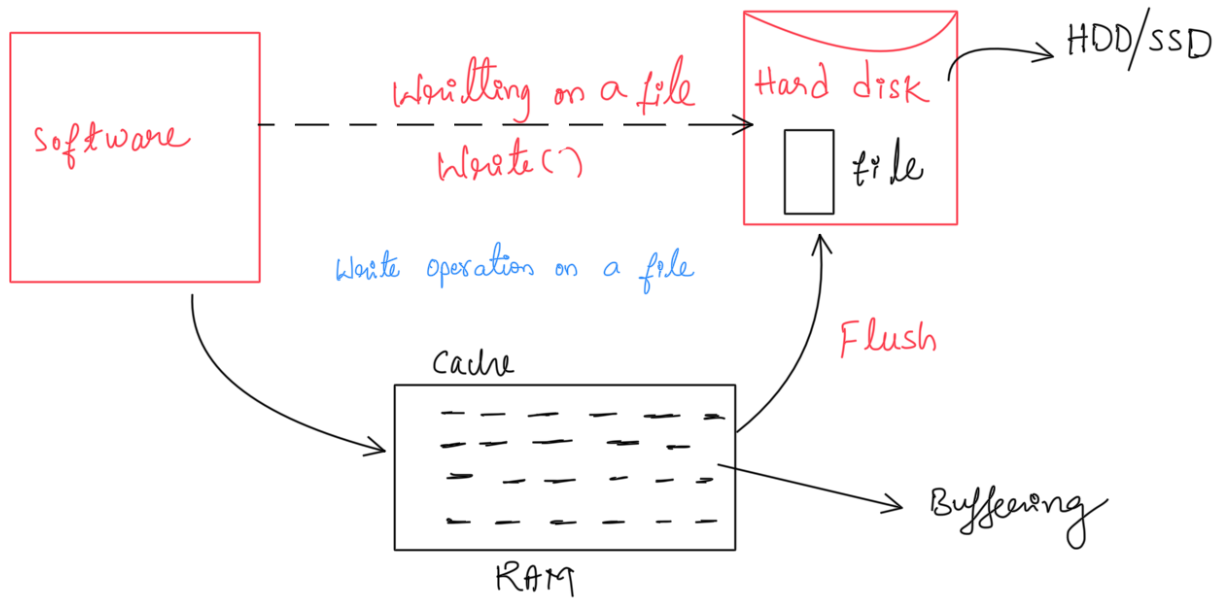
```
java
import java.util.HashSet;

class Main {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>();
        set.add("Apple");
        set.add("Banana");
        set.add("Apple"); // duplicate ignored
        System.out.println(set);
    }
}
```

☞ Teaches you uniqueness property **by practice**, not just theory.

✓ 5. Explore Source Code (Advanced)

- Many standard library classes in Java are **open source**.
- Example: Check `ArrayList.java` implementation → see how resizing works.
- This gives **deep insight** into why it's efficient.



What Happens when you write a file ?

When you "write to a file", the following sequence happens Internally

Step 1: Open the File

- The program asks the Operating System (OS) to Open a file
- The OS checks permission (read, write, execute)
- if allowed, the OS gives Back a file description / handle (like a ticket to access the file).

Step 2: Move File pointers

- Every File has a file pointers
- By default
 - write mode → pointer starts at the beginning (overwrites)
 - append mode → pointer starts at the end (adds content)

Step 3 : Write Data (Buffering)

- Data is first written to a Buffer in Memory (RAM)
- The Buffer improves Performance By grouping small writes
- Example : Writing "Hello" doesn't immediately hit the disk - It goes into a Buffer

Step 4 : OS Sends Data to Disk

- At Some Point, Buffer content stored in RAM is flushed to the disk :
 - Automatically when Buffer is full
 - When the program explicitly calls `flush()` or `close()`

The OS translates the data into blocks and stores it on the disk.

Step 5 : Close the File

- Closing ensures all buffers are flushed
- The OS frees resources (file descriptors)
- If you forgot to close, some data might not get written

2. Example in Java (Writing to File)

```
java
import java.io.FileWriter;
import java.io.IOException;

public class FileWriteExample {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("output.txt"); // open file in write mode
            writer.write("Hello, World!\n");
            writer.write("This is a file write operation.\n");
            writer.close(); // flush + close
            System.out.println("Successfully wrote to the file.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

🔑 Notes:

- `new FileWriter("output.txt")` → opens (or creates) file.
- `writer.write()` → writes to **buffer**, not directly disk.
- `writer.close()` → flushes buffer → commits data to disk.

◆ 4. Edge Cases

- If file doesn't exist → it's created (unless in append mode and directory doesn't exist).
- If file is locked → another process writing → error/exception.
- If power loss before `flush()` → buffer data may be lost.

◆ 5. Real-World Uses of File Writing

- Logging system (`log.txt`).
- Storing user data (configs, cache).
- Exporting reports (CSV, JSON).
- Database-like persistence.

◆ 3. Append Mode Example

```
java
FileWriter writer = new FileWriter("output.txt", true); // true = append mode
writer.write("Adding more data at the end!\n");
writer.close();
```

✅ Summary:

Writing to a file involves **opening** → **buffering** → **flushing** → **closing**. The OS handles permissions, buffering, and disk operations to make it efficient and reliable.

There are two types of Input/output

① Synchronous

Tasks run one after another, in sequence

Each operation must finish before the next one starts

The program waits for the result

② Asynchronous

Tasks can run independently without waiting

The program continues while the operation happens in the background

You usually get the result later via callbacks, futures or promises.

👉 Example:

Synchronous

```
java

System.out.println("Start");

// Writing to file synchronously
try (FileWriter writer = new FileWriter("data.txt")) {
    writer.write("Hello, World!");
} catch (Exception e) {
    e.printStackTrace();
}

System.out.println("End");
```

Output:

```
Start
End
```

java

Asynchronous

```
import java.util.concurrent.*;

public class AsyncExample {
    public static void main(String[] args) {
        System.out.println("Start");

        CompletableFuture.runAsync(() -> {
            try {
                Thread.sleep(2000); // simulate long task
                System.out.println("File written (async)");
            } catch (Exception e) {
                e.printStackTrace();
            }
        });

        System.out.println("End (does not wait for file write)");
    }
}
```

Possible Output:

```
Start
End (does not wait for file write)
File written (async)
```

👉 The "End" prints immediately, while file writing finishes later.

◆ 3. Key Differences

Feature	Synchronous	Asynchronous
Execution	One task at a time, sequential	Multiple tasks can overlap
Waiting	Program waits for each task	Program does not wait
Performance	Slower if tasks are I/O-heavy (e.g., network, file)	Faster for I/O-heavy, as CPU can do other work
Code Simplicity	Easier to write and debug	Harder (callbacks, futures, async APIs)
Best for	CPU-heavy tasks (math, sorting)	I/O-heavy tasks (network, DB, file read/write)

• Synchronous:

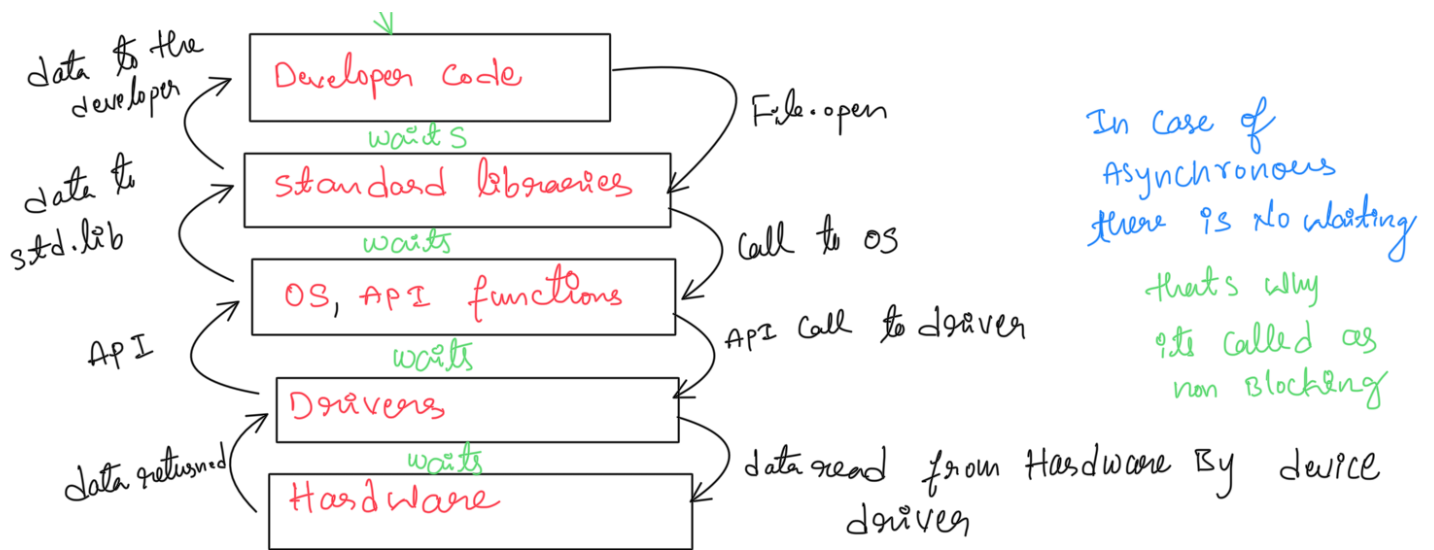
- Reading files line by line.
- Simple scripts.
- Database queries in single-threaded apps.

• Asynchronous:

- Web servers (handle 1000s of requests without blocking).
- UI applications (don't freeze when loading data).
- Messaging apps (send/receive in background).

Synchronous also called as Blocking

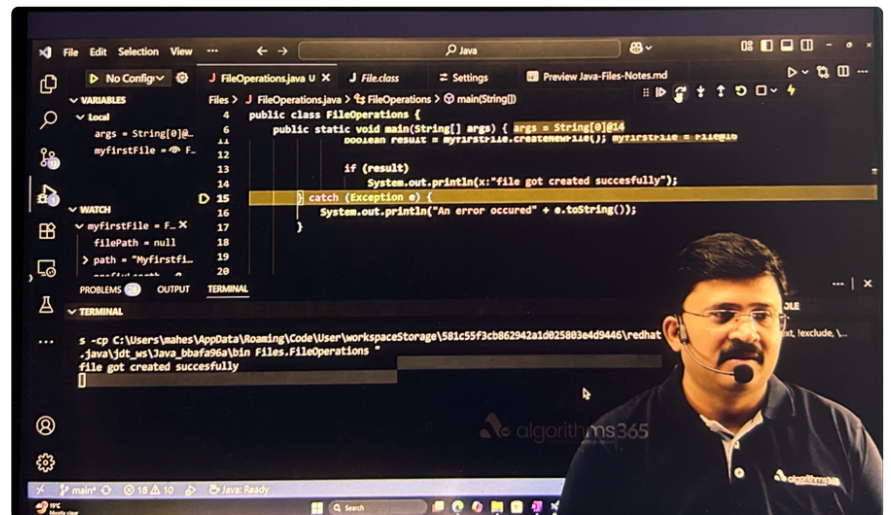
Asynchronous also called as non Blocking



In "wait time" all other processes are Blocked

Operations on a file

C → Create
R → Read
U → update
D → delete



create operation

I will Explore CRUD operations tomorrow